

Senior_ease_App

```
import { useState, useRef, useEffect } from "react";
import { Button } from "@components/ui/button";
import { Textarea } from "@components/ui/textarea";
import { ChatMessage } from "@components/ChatMessage";
import { VoiceButton } from "@components/VoiceButton";
import { QuickActions } from "@components/QuickActions";
import { EmergencyHelp } from "@components/EmergencyHelp";
import { streamChat } from "@utils/chatStream";
import { Send, Sparkles } from "lucide-react";
import { useToast } from "@hooks/use-toast";

type Message = { role: "user" | "assistant"; content: string };

const Index = () => {
  const [messages, setMessages] = useState<Message[]>([]);
  const [input, setInput] = useState("");
  const [isLoading, setIsLoading] = useState(false);
  const messagesEndRef = useRef<HTMLDivElement>(null);
  const { toast } = useToast();

  const scrollToBottom = () => {
    messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
  };

  useEffect(() => {
    scrollToBottom();
  }, [messages]);

  const sendMessage = async (messageText: string) => {
    if (!messageText.trim() || isLoading) return;

    const userMsg: Message = { role: "user", content: messageText.trim() };
    setMessages((prev) => [...prev, userMsg]);
    setInput("");
    setIsLoading(true);

    let assistantContent = "";
    const updateAssistant = (chunk: string) => {
      assistantContent += chunk;
      setMessages((prev) => {
        const last = prev[prev.length - 1];
        if (last?.role === "assistant") {
          return prev.map((m, i) => (i === prev.length - 1 ? { ...m, content: assistantContent } : m));
        }
      });
      return [...prev, { role: "assistant", content: assistantContent }];
    };

    streamChat(messageText, updateAssistant);
  };
};
```

```

    try {
      await streamChat({
        messages: [...messages, userMsg],
        onDelta: (chunk) => updateAssistant(chunk),
        onDone: () => setIsLoading(false),
        onError: (error) => {
          toast({
            title: "Error",
            description: error,
            variant: "destructive",
          });
          setIsLoading(false);
        },
      });
    } catch (e) {
      console.error(e);
      setIsLoading(false);
      toast({
        title: "Error",
        description: "Something went wrong. Please try again.",
        variant: "destructive",
      });
    }
  };

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    sendMessage(input);
  };

  const handleVoiceTranscript = (text: string) => {
    setInput(text);
  };

  return (
    <div className="flex flex-col h-screen bg-background">
      {/* Header */}
      <header className="bg-card border-b-2 border-border shadow-sm py-6 px-6">
        <div className="max-w-4xl mx-auto flex items-center justify-between gap-4">
          <div className="flex items-center gap-3">
            <Sparkles className="h-10 w-10 text-primary" />
            <h1 className="text-3xl font-semibold text-foreground">Your
AI Assistant</h1>
          </div>
          <EmergencyHelp />
        </div>
      </header>

      {/* Messages */}
      <main className="flex-1 overflow-y-auto px-6 py-8">
        <div className="max-w-4xl mx-auto">
          {messages.length === 0 ? (

```

```

        <div className="text-center mb-12">
            <h2 className="text-4xl font-semibold mb-4 text-foreground">
                Hello! How can I help you today?
            </h2>
            <p className="text-xl text-muted-foreground mb-12">
                I'm here to assist you with any questions you have.
                Choose a topic below or type your own question.
            </p>
            <QuickActions onSelectPrompt={sendMessage} />
        </div>
    ) : (
        messages.map((msg, idx) => <ChatMessage key={idx}
        role={msg.role} content={msg.content} />)
    )}
    <div ref={messagesEndRef} />
</div>
</main>

{/* Input */}
<footer className="bg-card border-t-2 border-border shadow-lg">
    <form onSubmit={handleSubmit} className="max-w-4xl mx-auto p-6">
        <div className="flex gap-3 items-end">
            <Textarea
                value={input}
                onChange={(e) => setInput(e.target.value)}
                placeholder="Type your question here..."
                className="flex-1 min-h-[80px] text-lg resize-none rounded-2xl"
                disabled={isLoading}
                onKeyDown={(e) => {
                    if (e.key === "Enter" && !e.shiftKey) {
                        e.preventDefault();
                        handleSubmit(e);
                    }
                }}
            />
            <div className="flex gap-2">
                <VoiceButton onTranscript={handleVoiceTranscript}
                disabled={isLoading} />
                <Button type="submit" size="lg" disabled={isLoading ||
                !input.trim()} className="shrink-0">
                    <Send className="h-6 w-6" />
                </Button>
            </div>
        </div>
        <p className="text-sm text-muted-foreground mt-3 text-center">
            Press Enter to send, Shift+Enter for new line
        </p>
    </form>
</footer>
</div>
);
};

```

```
export default Index;
```